Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Zaid Alsaheb IE University, School of Science & Technology

Supervised by Professor Raul Perez Pelaez

Abstract

We evaluate WebGPU as a compute platform for scientific simulation by implementing and benchmarking a complete Barnes–Hut gravitational N-body solver. The solver constructs and traverses a Linear Bounding Volume Hierarchy (LBVH) entirely on the GPU each timestep using the parallel method of Karras, combined with a symplectic leapfrog integrator. A single C++/WGSL codebase targets both native desktop execution and browser deployment via WebAssembly. We characterise performance across four WebGPU implementations (wgpu-native, Dawn, Chrome, and Safari) on an Apple M2 and compare against a native Metal Barnes–Hut baseline to isolate the cost of the WebGPU abstraction layer. At $N = 1000$ the abstraction layer imposes a 2.0 $\times$ overhead relative to native Metal; at $N \geq 5000$ the WebGPU solver outperforms the baseline, reaching 2.9$\times$ faster at $N = 100000$. Force evaluation accounts for 94–99% of total step time, with LBVH construction staying below 0.35 ms at all tested particle counts. Browser execution via Emscripten introduces a scaling overhead that narrows from 2.0$\times$ to 1.4$\times$ as GPU compute dominates, preserving the qualitative scaling behaviour seen in native execution. The 32-bit floating-point precision of current WebGPU implementations, together with the tree force approximation, constrains numerical fidelity. The results show that WebGPU's compute shader model can support a hierarchical N-body solver with parallel tree construction, and that the abstraction overhead is low enough for interactive galactic dynamics simulation without specialised hardware or vendor-locked APIs.

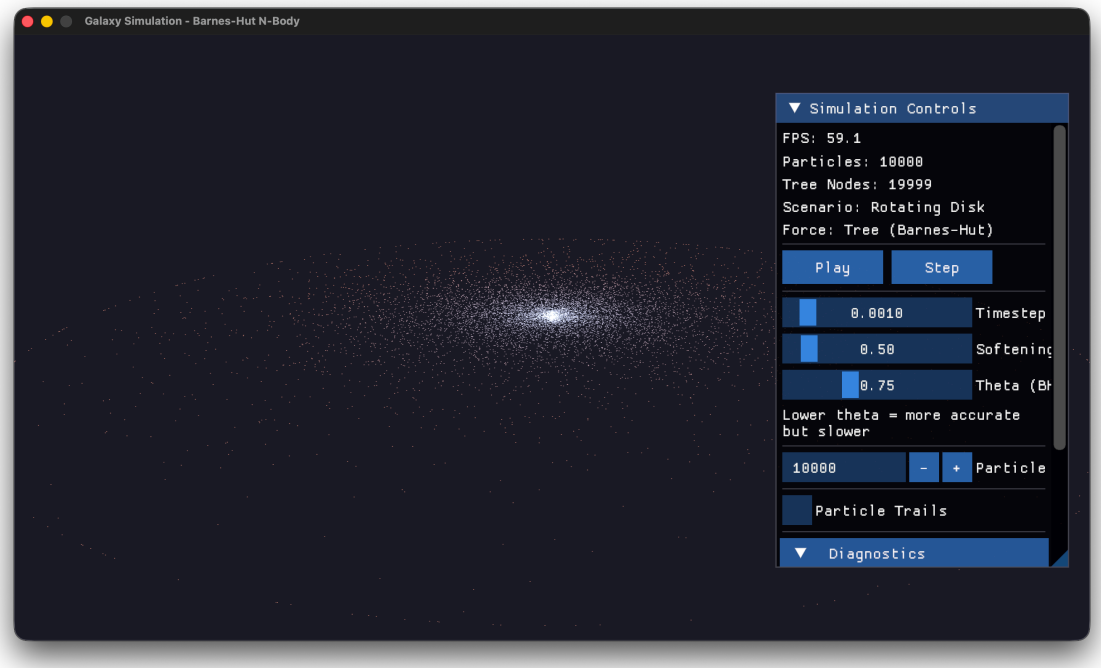

Figure 1: The interactive simulator running a rotating exponential disk with real-time ImGui controls for simulation parameters, timing diagnostics, and rendering options.

Key Results

The solver scales from **5.86 ms/step at $N = 1000$** to **180.11 ms/step at $N = 100000$**. Force evaluation accounts for **94–99% of total step time**; the entire LBVH tree construction pipeline stays **below 0.35 ms** at all particle counts. The bottleneck is the BVH traversal shader, not the tree build.

WebGPU vs Native Metal

Compared against UniSim, a native Metal Barnes–Hut solver on the same Apple M2, **WebGPU outperforms native Metal at $N \geq 5000$**, reaching **2.9× faster at $N = 100000$** (180 ms vs 517 ms). The advantage comes from the fully GPU-resident LBVH pipeline, which eliminates CPU–GPU coordination overhead.

N	Metal (ms)	WebGPU (ms)	Ratio
1,000	2.94	5.86	2.0× slower
5,000	10.09	7.35	**1.4× faster**
10,000	21.35	11.00	**1.9× faster**
50,000	121.77	65.46	**1.9× faster**
100,000	516.78	180.11	**2.9× faster**

Table 1: WebGPU (wgpu-native) vs native Metal (UniSim) on Apple M2. WebGPU overtakes Metal above $N = 5000$.

Cross-Implementation and Browser Performance

Testing four WebGPU backends on identical hardware shows **large variation**: Dawn has the lowest dispatch overhead (**1.40 ms** at $N = 1000$) while wgpu-native scales best (**180 ms** at $N = 100000$). In the browser, Chrome is **faster than native at $N = 1000$** (4.87 ms vs 5.86 ms) but scales worse, settling at **1.4× overhead by $N = 100000$**.

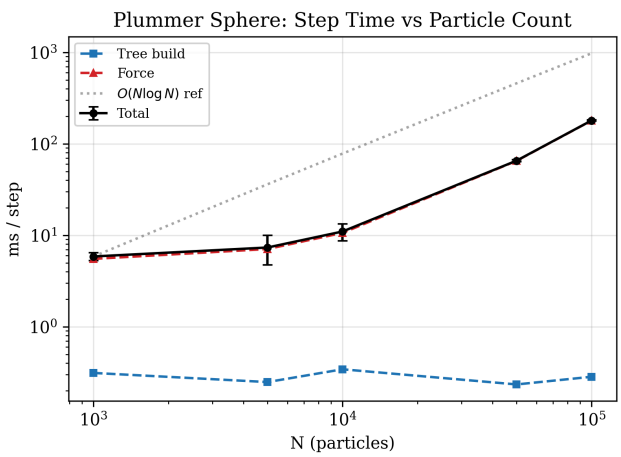

Figure 2: Runtime vs particle count (log-log).

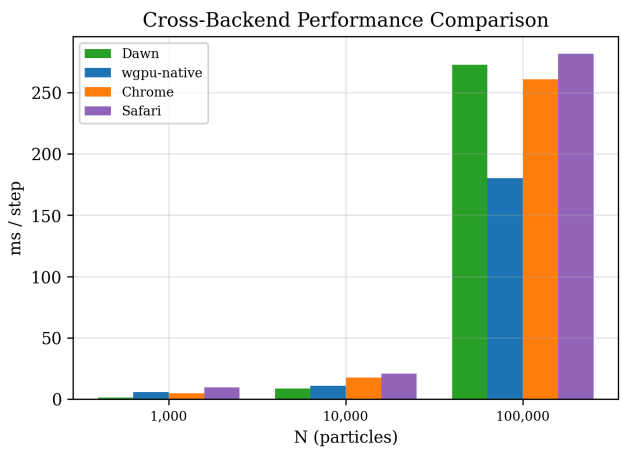

Figure 3: Four WebGPU implementations compared.